

In this assignment, you will implement Sequence-to-Sequence (Seq2Seq) workflow for text summarization using the CNN/DailyMail dataset.

1. Establish Baselines: Quantify the 'Context Bottleneck' using Lead-3 heuristics.
2. Fine-Tuning with Attention: Adapt a pre-trained BART or T5 model to the news domain using the Trainer API.
3. Interpretability Study: Visualize Attention Heatmaps to prove that the model can 'attend' to relevant facts regardless of their position in the text

This assignment builds on Module 4 (RNNs & LSTMs) and emphasizes the transition from manual architectural builds to industry-standard transfer learning

Bridging the Gap: From Keras to Hugging Face

In Week 4, you built models by "wiring" layers together manually using Keras. You defined the input, hidden layers, and output, and then you told the model how to learn using `.compile()` and `.fit()`.

This week, we are moving to **Transfer Learning** using the Hugging Face `Trainer` API. This can feel like a jump in complexity because the "training loop" is no longer visible—it is abstracted away. Use the mapping below to translate what you know (Keras) to what you are about to use (Hugging Face).

- **The Model:** In Keras, you built the model step-by-step. In Hugging Face, you download a pretrained model (`AutoModelForSeq2SeqLM.from_pretrained`)
- **Configuration:** Instead of `model.compile()` , you set hyperparameters in a config object called `Seq2SeqTrainingArguments`
- **The Loop:** Instead of `model.fit()` , you initialize a `Seq2SeqTrainer` object and call `trainer.train()` . The Trainer automatically handles dynamic padding, learning rate decay, and GPU optimization.

Part 1: Preprocessing and Dataset Preparation (20 Points)

Use the code below to download the CNN/DailyMail dataset (3.0.0) Standardized Sampling of 2,000 training records, 500 validation and 500 test records

Initialize a pre-trained tokenizer (e.g. `AutoTokenizer`) matching your selected Transformer model (Bart or T5)

Convert the processed records into a format ready for a Keras-integrated data loader

```
In [1]: ## Student Code Required ##  
        # Instruction: Initialize the tokenizer and determine the optimal sequence  
        # lengths for articles vs. highlights.  
        # You can enforce a 2,000-sample training limit to assist with execution  
  
        # ===== Part 1: Dataset & Tokenization =====
```

```

from transformers import AutoTokenizer

model_checkpoint = "facebook/bart-base"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
input_max_len = 512
label_max_len = 128

def preprocess_function(examples):
    # Category A Blanks: Define truncation and padding strategies
    model_inputs = tokenizer(examples["article"],
                             max_length=input_max_len, # Choose 512 for articles
                             truncation=True,
                             padding="max_length")

    # Updated: as_target_tokenizer was deprecated
    labels = tokenizer(text_target = examples["highlights"],
                       max_length=label_max_len, # Choose 128 for summaries
                       truncation=True,
                       padding="max_length")

    model_inputs["labels"] = labels["input_ids"]

    return model_inputs

```

```

/home/water/Grad School/DSE6212/.venv/lib/python3.12/site-packages/tqdm/auto.py:21: TqdmWarning:
IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/
en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

```

```

In [2]: from datasets import load_dataset

raw_dataset = load_dataset("cnn_dailymail", "3.0.0")

```

```

In [3]: # Student Task: Map the preprocessing function to the standardized subsets
# (2000 train / 500 val / 500 test)
train = raw_dataset["train"].shuffle(seed=42).select(range(2000))
val = raw_dataset["validation"].shuffle(seed=42).select(range(500))
test = raw_dataset["test"].shuffle(seed=42).select(range(500))

train_tokenized = train.map(preprocess_function, batched=True)
val_tokenized = val.map(preprocess_function, batched=True)
test_tokenized = test.map(preprocess_function, batched=True)

```

```

Map: 100%|██████████| 2000/2000 [00:01<00:00, 1754.72 examples/s]
Map: 100%|██████████| 500/500 [00:00<00:00, 2686.58 examples/s]
Map: 100%|██████████| 500/500 [00:00<00:00, 2513.43 examples/s]

```

Exercise:

1. In the code, we set the max sequence length for articles to 512. Why don't we set it to 2,000 to capture the whole article? Explain how the Self-Attention mechanism's memory requirement scales with sequence length (Hint: is it linear or quadratic?).

The max sequence length was limited as a result of the multiplicative scaling of the attention mechanism. The attention matrix is $\text{max_sequence_length} \times \text{max_sequence_length}$ in size, which results in massive RAM requirements. This grows quadratically with sequence length.

2. Discuss how excessive truncation might lead to a loss of the 'Lead-3' information.

Lead-3 information refers to the most important contextual information being present at the beginning of an article or body of writing. Excessive truncation could begin to remove this critical contextual information, reducing the information from which a model can learn.

Part 2: Baseline Performance Benchmarking (20 Points)

```
In [4]: ## Student Code Required ##
        # Instruction: Implement the Lead-3 heuristic.
        # Then, evaluate both Lead-3 and a Zero-Shot BART model to
        # establish a quantitative 'floor' for performance.

import nltk
import torch

#download required data for sentence tokenizing
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)

# Load model directly
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)

# Move to GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)
```

```
Loading weights: 100%|██████████| 259/259 [00:00<00:00, 773.22it/s, Materializing param=model.shared.weight]
```

```

Out[4]: BartForConditionalGeneration(
  (model): BartModel(
    (shared): BartScaledWordEmbedding(50265, 768, padding_idx=1)
    (encoder): BartEncoder(
      (embed_tokens): BartScaledWordEmbedding(50265, 768, padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 768)
      (layers): ModuleList(
        (0-5): 6 x BartEncoderLayer(
          (self_attn): BartAttention(
            (k_proj): Linear(in_features=768, out_features=768, bias=True)
            (v_proj): Linear(in_features=768, out_features=768, bias=True)
            (q_proj): Linear(in_features=768, out_features=768, bias=True)
            (out_proj): Linear(in_features=768, out_features=768, bias=True)
          )
          (self_attn_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (activation_fn): GELUActivation()
          (fc1): Linear(in_features=768, out_features=3072, bias=True)
          (fc2): Linear(in_features=3072, out_features=768, bias=True)
          (final_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
      )
      (layernorm_embedding): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    )
    (decoder): BartDecoder(
      (embed_tokens): BartScaledWordEmbedding(50265, 768, padding_idx=1)
      (embed_positions): BartLearnedPositionalEmbedding(1026, 768)
      (layers): ModuleList(
        (0-5): 6 x BartDecoderLayer(
          (self_attn): BartAttention(
            (k_proj): Linear(in_features=768, out_features=768, bias=True)
            (v_proj): Linear(in_features=768, out_features=768, bias=True)
            (q_proj): Linear(in_features=768, out_features=768, bias=True)
            (out_proj): Linear(in_features=768, out_features=768, bias=True)
          )
          (activation_fn): GELUActivation()
          (self_attn_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (encoder_attn): BartAttention(
            (k_proj): Linear(in_features=768, out_features=768, bias=True)
            (v_proj): Linear(in_features=768, out_features=768, bias=True)
            (q_proj): Linear(in_features=768, out_features=768, bias=True)
            (out_proj): Linear(in_features=768, out_features=768, bias=True)
          )
          (encoder_attn_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (fc1): Linear(in_features=768, out_features=3072, bias=True)
          (fc2): Linear(in_features=3072, out_features=768, bias=True)
          (final_layer_norm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
      )
      (layernorm_embedding): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    )
  )
  (lm_head): Linear(in_features=768, out_features=50265, bias=False)
)

```

```

In [5]: def get_lead_3_baseline(text):
  try:
    sentences = nltk.sent_tokenize(text)
  except:
    # Fallback if NLTK fails or text is empty

```

```

        sentences = text.split(".")

    return " ".join(sentences[:3])

def get_zeroshot_summary(text, model, tokenizer):
    # Tokenize and move to device
    inputs = tokenizer(text, return_tensors="pt", max_length=input_max_len, truncation=True)
    inputs = {k: v.to(device) for k, v in inputs.items()}

    # Generate Summary IDs... Use 'no_grad' to save memory
    with torch.no_grad():
        summary_ids = model.generate(
            inputs["input_ids"],
            num_beams=4,
            max_length=128,
            min_length=30,
            early_stopping=True
        )

    # Decode IDs to Text
    summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
    return summary

```

```

In [6]: from tqdm import tqdm
import evaluate

rouge = evaluate.load('rouge')

# Lists to store results
lead_3_preds = []
zero_shot_preds = []
references = []

# Iterate through the val dataset
for i, example in enumerate(tqdm(val)):
    article = example["article"]
    highlights = example["highlights"]

    # Generate Lead-3
    lead_3_preds.append(get_lead_3_baseline(article))

    # Generate Zero-Shot
    zero_shot_preds.append(get_zeroshot_summary(article, model, tokenizer))

    # Store Reference
    references.append(highlights)

# Calculate Lead-3 Score
lead_3_results = rouge.compute(
    predictions=lead_3_preds,
    references=references
)
print(f"Lead-3 ROUGE-L: {lead_3_results['rougeL']:.4f}")

# Calculate Zero-Shot Score
zero_shot_results = rouge.compute(
    predictions=zero_shot_preds,
    references=references
)

```

```
)  
print(f"Zero-Shot ROUGE-L: {zero_shot_results['rougeL']:.4f}")
```

```
100%|██████████| 500/500 [05:48<00:00, 1.44it/s]
```

```
Lead-3 ROUGE-L: 0.2596
```

```
Zero-Shot ROUGE-L: 0.2513
```

Exercise:

1. Explain the 'Context Bottleneck.' Why do standard Seq2Seq models struggle to summarize articles when the most relevant information is buried in the middle of the text rather than the beginning?

In traditional Seq2Seq models like RNN or LSTM, the encoder layer distills the the sequence into a single, flat vector of a fixed size. The amount of information that can be compressed into this vector begins to reduce over time, resulting in less signal from contextually relevant information found later in the article. Even LSTM struggles; it can forget unneeded contextual inputs, but eventually the flat vector gets diluted with all of the additional sequential inputs.

Part 3: Transformer Fine-Tuning with Attention (40 Points)

```
In [8]: ## Student Code Required ##  
# Instruction: Configure the Seq2SeqTrainingArguments.  
# You must select the learning rate and epoch count that balances  
# convergence with the risk of catastrophic forgetting.  
  
# ===== Part 3: Fine-Tuning =====  
from transformers import Seq2SeqTrainingArguments, Seq2SeqTrainer, DataCollatorForSeq2Seq  
import torch  
  
training_args = Seq2SeqTrainingArguments(  
    output_dir="./results",  
    learning_rate=5e-5,      # Hint: 5e-5 is standard for BART  
    num_train_epochs=5,      # Hint: 3 to 5 is sufficient  
    per_device_train_batch_size=8,  
    predict_with_generate=True,  
    fp16=torch.cuda.is_available(),  
)  
  
data_collator = DataCollatorForSeq2Seq(  
    tokenizer=tokenizer,  
    model=model,  
    padding="longest",  
    max_length=input_max_len  
)  
  
# Category B Scaffold: Initialization of the Trainer API  
trainer = Seq2SeqTrainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_tokenized,  
    eval_dataset=test_tokenized,  
    data_collator=data_collator  
)
```

```
# Train the model
trainer.train()
```

[1250/1250 01:30, Epoch 5/5]

Step Training Loss

500	0.442845
1000	0.289701

```
Writing model shards: 100%|██████████| 1/1 [00:00<00:00, 1.51it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00, 1.37it/s]
Writing model shards: 100%|██████████| 1/1 [00:00<00:00, 1.25it/s]
```

```
Out[8]: TrainOutput(global_step=1250, training_loss=0.3397682098388672, metrics={'train_runtime': 90.8508, 'train_samples_per_second': 110.071, 'train_steps_per_second': 13.759, 'total_flos': 3048682291200000.0, 'train_loss': 0.3397682098388672, 'epoch': 5.0})
```

Exercise:

1. How does the `predict_with_generate` argument differ from standard training?

The `predict_with_generate` argument is critical for sequence to sequence generative training, as the model is fed a ground truth sequence input and attempts to predict the next sequence output token. This is the basis used to compute the losses, so the model is updated with how correct it's generated sequence prediction is.

2. We configured the model to use **Beam Search** for generating summaries. Contrast Beam Search with **Greedy Search**. Why is Greedy Search highly likely to fail in a sequence-to-sequence summarization task, and how does Beam Search fix this?

Greedy search is a token based probabilistic mechanism, selecting only the most probable next token in the sequence. This is dangerous in sequence to sequence generation because it cannot backtrack and interpret the impact the next token has on the prior sequence, which can lead to erroneous dead ends in the generated sequence. Beam search, in contrast, keeps track of the N-highest probability sequences, which allows for more coherent sequential generation. Additionally, there are multiple paths through which the model can explore, selecting the most optimal for the final generated sequence.

Part 4: Model Evaluation & Interpretability (20 points)

```
In [9]: ## Student Code Required ##
# Instruction: Generate a comparison table of ROUGE scores.
# Then, execute the Heatmap visualization for ONE article and
# identify the 'Alignment' between summary tokens and input tokens

# ===== Part 4: Attention Visualization =====
# Student Task: Use aggregate_cross_attention() to visualize weights.
```

```
In [10]: import pandas as pd
import numpy as np
```

```

print("Generating predictions with the fine-tuned model...")
predictions = trainer.predict(val_tokenized)

# Get just the prediction matrices
preds = np.array(predictions.predictions)

# If the model returned raw logits (3D array) instead of token IDs,
# grab the most likely token ID using argmax.
if len(preds.shape) == 3:
    preds = np.argmax(preds, axis=-1)

# Replace -100 with the actual pad_token_id. This was the only solution I was able to find for ar
preds = np.where(preds != -100, preds, tokenizer.pad_token_id)
labels = np.where(predictions.label_ids != -100, predictions.label_ids, tokenizer.pad_token_id)

# Decode the cleaned predictions and labels
decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
decoded_labels = tokenizer.batch_decode(predictions.label_ids, skip_special_tokens=True)

# Calculate Fine-Tuned ROUGE Scores
fine_tuned_results = rouge.compute(predictions=decoded_preds, references=decoded_labels, use_stemmer=True)

# Generate the Comparison Table
data = {
    "Model / Heuristic": ["Lead-3 Baseline", "Zero-Shot bart-base", "Fine-Tuned bart-base"],
    "ROUGE-1": [lead_3_results['rouge1'], zero_shot_results['rouge1'], fine_tuned_results['rouge1']],
    "ROUGE-L": [lead_3_results['rougeL'], zero_shot_results['rougeL'], fine_tuned_results['rougeL']]
}

df_scores = pd.DataFrame(data)
print("\n--- Final ROUGE Score Comparison ---")
print(df_scores.to_string(index=False))

```

Generating predictions with the fine-tuned model...

--- Final ROUGE Score Comparison ---

Model / Heuristic	ROUGE-1	ROUGE-L
Lead-3 Baseline	0.405056	0.405056
Zero-Shot bart-base	0.393106	0.251342
Fine-Tuned bart-base	0.250166	0.204432

```

In [11]: import matplotlib.pyplot as plt
import seaborn as sns
import torch

def aggregate_cross_attention(model, tokenizer, text, device="cuda"):
    """
    Runs a forward pass and extracts cross-attention weights for visualization.
    """
    model.eval()

    # Disable SDPA and force eager attention calculation... I really had to dig to find this solution
    # This forces the model to use the traditional math that saves the weights, resolving an error
    model.config._attn_implementation = "eager"
    model.config.output_attentions = True

    # Tokenize input
    inputs = tokenizer(text, return_tensors="pt", max_length=input_max_len, truncation=True)
    input_ids = inputs["input_ids"].to(device)

    # Generate summary and return attentions

```



```

with torch.no_grad():
    outputs = model.generate(
        input_ids,
        max_length=50,
        output_attentions=True,
        return_dict_in_generate=True,
        early_stopping=True
    )

    # Extract generated tokens
    generated_ids = outputs.sequences[0]
    generated_tokens = tokenizer.convert_ids_to_tokens(generated_ids, skip_special_tokens=False)
    input_tokens = tokenizer.convert_ids_to_tokens(input_ids[0], skip_special_tokens=False)

    seq_len = len(generated_tokens) - 1
    attention_matrix = torch.zeros(seq_len, len(input_tokens))

    for step in range(seq_len):
        last_layer_attn = outputs.cross_attentions[step][-1]
        avg_head_attn = last_layer_attn[0].mean(dim=0).squeeze(0)
        attention_matrix[step] = avg_head_attn.cpu()

    return attention_matrix.numpy(), input_tokens, generated_tokens[1:]

```

```

In [17]: # ===== Part 4: Attention Visualization =====

# Select one article from the validation set
sample_article = raw_dataset["validation"]["article"][100]

# Get the attention matrix and tokens
attention_weights, x_tokens, y_tokens = aggregate_cross_attention(model, tokenizer, sample_article)

x_tokens = [token.strip('Ġ') for token in x_tokens]
y_tokens = [token.strip('Ġ') for token in y_tokens]

# Plot the Heatmap
# Limit to the first 25 generated tokens and first 40 input tokens to keep the chart readable
plt.figure(figsize=(25, 18))
sns.heatmap(
    attention_weights[:70,:100],
    xticklabels=x_tokens[:100],
    yticklabels=y_tokens[:70],
    cmap="Blues",
    cbar_kws={'label': 'Attention Weight'}
)
plt.title("Cross-Attention: Summary Tokens vs. Input Article Tokens")
plt.xlabel("Input Article Tokens")
plt.ylabel("Generated Summary Tokens")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```



Exercise:

1. The Mechanism: Describe how the Attention Mechanism you visualized allows the decoder to "bypass" the bottleneck of a single fixed context vector used in standard RNNs.

The Attention Mechanism bypasses the context bottleneck by allowing the decoder to "look back" at the hidden states of all input tokens at every single step of generation. Instead of receiving one static context vector, the decoder calculates a weighted sum of the encoder's hidden states, assigning high attention weights to the specific source words most relevant to the word it is currently trying to generate.

2. Success Verification: In your heatmap above, identify a specific word in the summary that strongly attends to a relevant word in the source text. (e.g., "The summary word 'Apple' attended strongly to 'Cupertino' in the source").

In the heatmap above, the words "resignation letter" strongly attend to "Pichet" who is the person that wrote the resignation letter.

It is easy to find examples where the model works. As a Data Scientist, your job is to find where it breaks.

Task: Failure Mode Analysis Run the visualization code above for a few different samples (change the index `i`) until you find an instance of **Attention Failure** or **Hallucination**.

Analysis Prompt: Locate a summary where the model generated a word that was **not** supported by the source text, or where it missed a key fact.

- **The Hallucination:** What word did the model generate that wasn't in the source?
- **The Heatmap:** Look at the attention column for that specific hallucinated token.
 - Did it attend to an irrelevant word (Misguided Attention)?
 - Was the attention "scattered" (uniform faint weight across all words)?
- **Implication:** Discuss what this failure implies for using such models in high-stakes fields like Financial News or Medical Summarization.

```
In [ ]: # ===== Part 4: Attention Visualization =====

# Select one article from the validation set
sample_article = raw_dataset["validation"]["article"][189]

# Get the attention matrix and tokens
attention_weights, x_tokens, y_tokens = aggregate_cross_attention(model, tokenizer, sample_article)

x_tokens = [token.strip('Ġ') for token in x_tokens]
y_tokens = [token.strip('Ġ') for token in y_tokens]

# Plot the Heatmap
plt.figure(figsize=(25, 18))
sns.heatmap(
    attention_weights[:70,:100],
    xticklabels=x_tokens[:100],
    yticklabels=y_tokens[:70],
    cmap="Blues",
    cbar_kws={'label': 'Attention Weight'}
)
plt.title("Cross-Attention: Summary Tokens vs. Input Article Tokens")
plt.xlabel("Input Article Tokens")
plt.ylabel("Generated Summary Tokens")
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

